React 학습 노트

정리일 2026-11-09 · Week11-12 AI 활용

학습 목표는 좋아요 버튼 하나를 예로 들어, 상태가 바뀐 뒤 브라우저 화면에 반영되기까지의 과정을 설명하고 미니 React 구현에 연결하는 것이다.

1. 컴포넌트, 상태, DOM

버튼 하나에서 본 React

- **컴포넌트**는 화면의 한 부분을 정의한다. Counter는 버튼을 반환한다.
- **Props**는 부모가 넘기는 입력이다. step={2}라면 클릭할 때 2씩 더한다. **State**인 count는 현재 숫자다.
- useState(0)의 0은 초기값이다. 같은 컴포넌트가 유지되는 동안 재렌더링하면 React가 기억한 현재 값을 읽는다.
- 버튼에 단 onClick은 그 버튼의 클릭을 처리한다. 클릭 직후 실행되는 것은 이벤트 핸들러이며, Counter는 상태 업데이트 뒤 렌더링 과정에서 다시 실행된다.

DOM과 Virtual DOM

DOM은 브라우저가 관리하는 문서 트리다. window는 현재 창이나 탭을, document는 그 안의 문서를 다루는 객체다.

```
document
  html
    body
      button#like
        "좋아요 10"
```

document.querySelector("#like")로 버튼을 찾고 button.textContent에 새 글자를 넣으면 실제 DOM이 바뀐다. 일반 변수에 1을 더해도 기존 DOM은 그대로다. 이 예제에서 MutationObserver의 childList 옵션으로 버튼 안의 텍스트 노드 교체를 관찰할 수 있다.

Virtual DOM은 화면 구조를 JavaScript 데이터로 표현하는 개념이다. 학습용 표현의 예는 아래와 같다.

```
{ type: "button", props: { id: "like" }, children: ["좋아요 10"] }
```

렌더링으로 새 구조를 계산하고, **Diff**로 이전·새 결과를 비교하고, **Commit**에서 필요한 변경을 실제 DOM에 적용한다. 같은 버튼의 글자만 달라지면 버튼을 유지할 수 있고, 같은 위치의 요소 타입이 바뀌면 교체할 수 있다. 브라우저는 변경된 DOM을 바탕으로 화면을 그린다.

2. 서버 요청과 화면 갱신

좋아요 버튼의 실행 순서

서버 응답을 받은 뒤 화면을 갱신하는 예시다.

```
버튼 클릭
→ 이벤트 핸들러가 서버에 증가 요청
→ 서버가 DB의 값을 1 올리고 새 값을 응답
→ 이벤트 핸들러가 setCount(새 값) 호출
→ React가 Counter를 다시 실행해 새 화면 구조 계산
→ Diff로 이전·새 결과 비교 → Commit에서 DOM 변경
→ 브라우저가 화면에 표시
```

- **AJAX**는 새로고침 없이 서버와 데이터를 주고받는 방식이다. 예제에서는 JavaScript가 fetch()를 호출하면 브라우저가 요청을 보낸다.
- 클릭 처리 코드가 **어느 게시물**을 수정할지 정한다. Diff는 **화면의 어느 부분**에 변경을 적용할지 찾는다. 좋아요 숫자 하나의 위치를 이미 안다면 직접 DOM을 바꿀 수도 있다.
- 좋아요가 9에서 10이 되면 숫자 변경과 함께 인기 배지가 추가될 수 있다. React에서는 각 상태에서 보여줄 화면을 작성하면, 렌더링 결과에서 두 변경을 찾을 수 있다.

상태를 어디에 두는가

- **React 상태**는 현재 화면에 쓸 값이다. 예를 들어 setIsOpen(true)만으로 댓글 작성창을 추가할 수 있다. 서버 요청 없이도 렌더링과 DOM 반영이 일어난다.
- **DB**는 게시글과 좋아요처럼 이후에도 보존할 데이터를 맡는다. 페이지를 새로고침하면 React는 초기 상태로 시작하고, 서버에서 저장된 값을 받아 화면 상태에 반영할 수 있다.
- **URL과 브라우저 방문 기록**으로 목록 페이지를 복원할 수 있다. /posts?page=2에서 뒤로 돌아와 /posts?page=1이 되면 앱은 1페이지에 해당하는 화면을 보여준다.
- 작성창이 트리에서 제거되면 그 안의 useState는 다시 열 때 초기화된다. 작성 내용을 유지하려면 계속 남아 있는 상위 컴포넌트에 두거나, 필요한 기간에 맞춰 별도로 저장한다.

3. 실습 현황과 다음 학습

CLI에서 만든 카운터

- app.py는 Flask 서버와 SQLite를 사용한다. GET /api/count로 값을 읽고, POST /api/count/increment로 값을 올린다.
- templates/index.html과 static/style.css는 화면을 구성한다. static/app.js는 최초 조회, 증가 요청, textContent 갱신, 요청 중 버튼 비활성화, 오류 표시를 담당한다.
- CLI 기록에 따르면 JavaScript 문법 검사와 모의 DOM·fetch 테스트는 통과했다. Flask 서버 실행과 실제 브라우저 동작은 해당 기록에서 검증되지 않았다.
- 현재 카운터는 일반 JavaScript로 DOM을 직접 갱신한다. 미니 React의 상태 저장, 렌더링, Diff는 다음 구현 범위다.

어디서 이어갈까

1. **Phase 1**에서 DOM 변경 관찰을 실습하고, Reflow·Repaint의 차이와 Diff 핵심 케이스 5가지를 확인한다.
2. **Phase 2**에서 컴포넌트 함수가 다시 실행되어도 상태를 기억하는 방법을 설계한다. setState가 새 값을 저장하고 화면 갱신을 요청하는 경로를 연결한다. Batching은 선택 과제다.
3. 통합 시연 페이지에서 클릭부터 DOM 변경까지 설명한다. 11/12(목) 발표는 미니 React, 11/19(목) 발표 계획은 미니 Redis·미니 DBMS다.

이어서 확인할 질문은 하나다. **DB에 좋아요 11이 저장된 채 새로고침하면, 서버 조회 전의 초기 상태와 조회 뒤 화면에 표시할 값은 각각 무엇인가?**

AI 활용 방식

먼저 동작 순서를 내 말로 설명하고, AI에게 빠진 단계나 오해 한 가지를 확인받는다. 작은 예제나 테스트로 확인한 뒤, 발표에서는 처음 몰랐던 점과 검증 방법을 AI 없이 다시 설명한다.

출처 · Week11-12의 «AI 활용» 학습 채팅과 CLI의 test-react/STUDY_NOTES.md.